

UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL - VILLA MARIA



MATEMATICAS – PROGRAMACIÓN

Alumnos:

Borsatto, Federico

Gallo, Valentina

Moreno, Pedro

Souto, Matías

Zoia, Tobias

ÍNDICE GENERAL

Consigna 1 - Validación de usuarios y análisis de consistencia del sistema=====Pág. 3

- **Parte A:** Análisis con conjuntos.....Pág. 3
- **Parte B:** Lógica proposicional.....Pág. 6
- **Parte C:** Interpretación.....Pág. 8

Consigna 2 - Análisis y optimización de costos de desarrollo=====Pág. 9

- **Parte A:** Análisis matemático.....Pág. 9
- **Parte B:** Implementación.....Pág. 13
- **Parte C:** Análisis.....Pág. 15

Consigna 3 - Análisis de rendimiento en un sistema distribuido=====Pág. 15

- **Parte A:** Comprensión del modelo.....Pág. 15
- **Parte B:** Implementación en Python.....Pág. 16
- **Parte C:** Análisis mediante producto matricial.....Pág. 17
- **Parte D:** Interpretación crítica.....Pág. 17
- **Parte E:** Propiedades de la matriz.....Pág. 17
- **Parte F:** Análisis aplicado.....Pág. 18

=====

=====

LINK A VIDEO DE YOUTUBE: <https://www.youtube.com/watch?v=GVAaJMach18>

LINK A REPOSITORIO GITHUB: <https://github.com/TobiasZoia/TPI-MATE-PROG>

NUESTROS USUARIOS DE GITHUB:

valentinagallo

morenopedrog

Matiassouto559

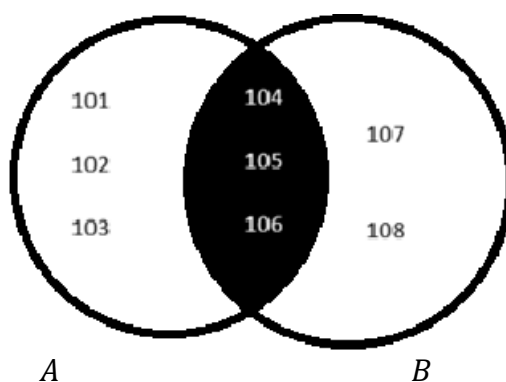
Federico Borsatto

TobiasZoia

Consigna 1 - Parte A

Se aplica la **intersección** ($A \cap B$). Buscamos los elementos que se repiten y están presentes en el conjunto A y en el conjunto B al mismo tiempo.

- $A = \{101, 102, 103, 104, 105, 106\}$
- $B = \{104, 105, 106, 107, 108\}$
- **Cálculo:** $A \cap B = \{104, 105, 106\}$

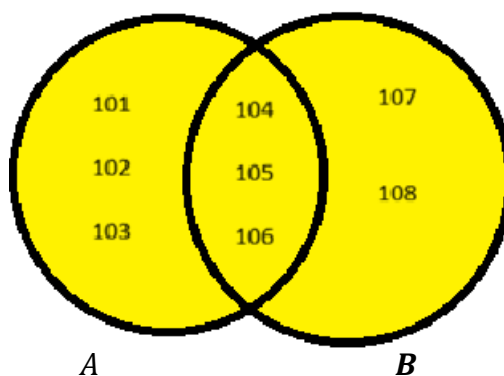


Los elementos 104, 105 y 106 pertenecen a ambas plataformas (son la *intersección*)

Usuarios que utilizan al menos una plataforma $A \cup B$

Se aplica la **unión** $A \cup B$ Consiste en unificar todos los elementos de A junto con todos los de B en una sola lista, anotando una sola vez aquellos que coinciden.

- **Cálculo:** $\{101, 102, 103, 104, 105, 106\} \cup \{104, 105, 106, 107, 108\}$
- **Resultado:** $A \cup B = \{101, 102, 103, 104, 105, 106, 107, 108\}$

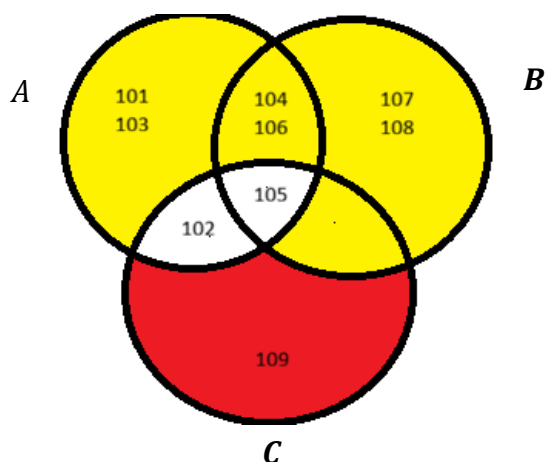


La unión $A \cup B$ incluye todos los elementos de A y B anotando una vez los que coinciden (intersección)

Usuarios de la plataforma, pero sin errores $(A \cup B) - C$

Se aplica una **diferencia**. Partimos de la unión total obtenida en el paso anterior $A \cup B$ y le **removemos** cualquier ID que figure dentro del conjunto de errores C .

- Unión base: {101, 102, 103, 104, 105, 106, 107, 108}
- Conjunto a restar C : {102, 105, 109}
- **Cálculo:** Eliminamos el 102 y el 105. El resultado es {101, 103, 104, 106, 107, 108}

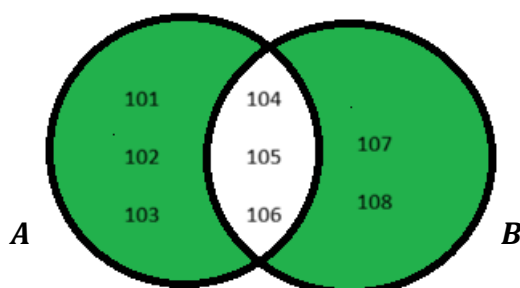


Partimos de la unión total $(A \cup B)$ y restamos el conjunto C . El número 102 $A \cap C$ y el 105 $A \cap B \cap C$ se remueven de la unión. El número 109 aunque es error. Se ignora por que no estaba en la union

Usuarios que utilizan exclusivamente una sola plataforma $A \Delta B$

Se aplica la **diferencia simétrica** (Δ). Representa a los usuarios que pertenecen a un canal o al otro, pero **no a ambos simultáneamente**. Equivale a tomar la unión y restarle la intersección central.

- Unión de los accesos: {101, 102, 103, 104, 105, 106, 107, 108}
- Intersección a remover: {104, 105, 106}
- **Cálculo:** $A \Delta B = \{101, 102, 103, 107, 108\}$



La diferencia simétrica incluye solo los elementos exclusivos de A y exclusivos de B, eliminando la intersección central

2)

Usuarios sin errores $(A \cup B) - C$

Para expresar tu resultado {101, 103, 104, 106, 107, 108}, la condición es que el usuario debe estar en la Plataforma A o en la Plataforma B (unión), pero **no** debe pertenecer al conjunto de errores C.

$$(A \cup B) - C = x \mid (x \in A \wedge x \in B) \wedge x \notin C$$

Errores puros $C - A \cup B$

Para expresar tu resultado {109}, la condición es que el ID debe pertenecer obligatoriamente al conjunto de errores C, pero **no** puede estar en la Plataforma A ni en la Plataforma B.

Usuarios en C pero no en $A \cup B$ ($C - A \cup B$)

Se aplica una **diferencia** -. El punto de partida es el conjunto de errores C y le **quitamos** cualquier ID que haya interactuado con las plataformas a través de la API o la Web $A \cup B$

- Conjunto de errores base C: {102, 105, 109}
- Filtro de usuarios en plataforma $A \cup B$: {101, 102, 103, 104, 105, 106, 107, 108}
- **Cálculo:** El 102 y el 105 se descartan por estar en la plataforma. Solo permanece el {109}



El diagrama incluye solo los elementos del conjunto C que no pertenecen a la unión de A y B (usuarios en plataforma). Los IDs 102 y 105 se eliminan por estar activos

Tabla de verdad

p	q	r	$(p \vee q)$	$(p \vee q) \wedge r$
V	V	V	V	V
V	V	F	V	F
V	F	V	V	V
V	F	F	V	F
F	V	V	V	V
F	V	F	V	F
F	F	V	F	F
F	F	F	F	F

Conclusión de la tabla: El resultado final demuestra que el argumento es una **contingencia** (posee valores verdaderos y falsos). El sistema clasificará al usuario como "Crítico" únicamente en los 3 casos donde la salida es **V**, es decir, cuando el usuario está activo en cualquier plataforma (p o q sean verdaderos) y, al mismo tiempo, experimenta fallas (r sea verdadero)

```

1  try:
2      A = [0]*6
3      B = [0]*5
4      C = [0]*3
5
6      print("Ingresar usuarios que ingresan desde la web: ")
7      for i in range(6):
8          A[i] = int(input())
9      print("Ingresar usuarios que ingresan desde la API")
10     for i in range(5):
11         B[i] = int(input())
12     print("Ingresar usuarios que generaron errores: ")
13     for i in range(3):
14         C[i] = int(input())
15     print()
16     print("CLASIFICACIÓN DE USUARIOS")
17     for i in range(6):
18         diferentes_en_C = 0
19         for j in range(3):
20             if A[i] != C[j]:
21                 diferentes_en_C+=1
22         if diferentes_en_C==3:
23             print(A[i], "= No critico")
24         if diferentes_en_C<3:
25             print(A[i], "= Critico")
26     for i in range(5):
27         diferentes_en_A=0
28         for j in range(6):
29             if B[i] != A[j]:
30                 diferentes_en_A+=1
31         if diferentes_en_A==6:
32             diferentes_en_C=0
33             for j in range(3):
34                 if B[i] != C[j]:
35                     diferentes_en_C+=1
36             if diferentes_en_C<3:
37                 print(B[i], "= Critico")
38             if diferentes_en_C==3:
39                 print(B[i], "= No critico")
40     for i in range(3):
41         diferentes_en_A=0
42         diferentes_en_B=0
43         for j in range(6):
44             if C[i] != A[j]:
45                 diferentes_en_A+=1
46         for j in range(5):
47             if C[i] != B[j]:
48                 diferentes_en_B+=1
49         if diferentes_en_A==6 and diferentes_en_B==5:
50             print(C[i], "= No critico")
51 except ValueError:
52     print("Error")

```

CONSIGNA 1 - PARTE C

¿Qué tipo de usuario representa mayor riesgo?

El tipo de usuario que representa mayor riesgo es aquel que está activo en alguna de las plataformas (A o B) y además pertenece al conjunto de errores C.

Son los usuarios que se encuentran en la intersección de los errores con las plataformas. Específicamente, el ID 102 (que está en $A \cap C$) y el ID 105 (que está en $A \cap B \cap C$). Utilizan la plataforma Web o API y al mismo tiempo generan errores en el sistema

¿Qué significa que un usuario esté en C pero no en $A \cup B$?

Significa que existe un ID registrado obligatoriamente en el conjunto de errores, pero que no ha interactuado con las plataformas a través de la API ni de la Web

El único ID que cumple esta condición es el 109. El sistema está arrojando un error para un usuario que ni siquiera accedió a la plataforma

Matemáticamente:

$$C - (A \cup B) = \{109\}$$

* ¿Qué decisión tomarían como equipo programador?

El equipo debería enfocarse tanto en:

- Resolver los problemas de los usuarios activos
- Como en investigar errores anómalos que podrían indicar fallas más profundas del sistema.

1 - Usuarios Críticos que acceden a la plataforma mediante la web y/o la API: La prioridad absoluta solucionar los errores que afectan a los ID 102 y 105.

2 – Solucionar el problema del ID 109: Se necesita descubrir por qué la base de datos reporta un error de un usuario inactivo y solucionarlo.

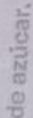
3 - Monitoreo Preventivo: Durante la implementación de cualquier solución, se debe garantizar que los usuarios (el conjunto de usuarios sin errores A u B - C) no se vean afectados.

Consigna 2 — Análisis y optimización de costos de desarrollo

Parte A

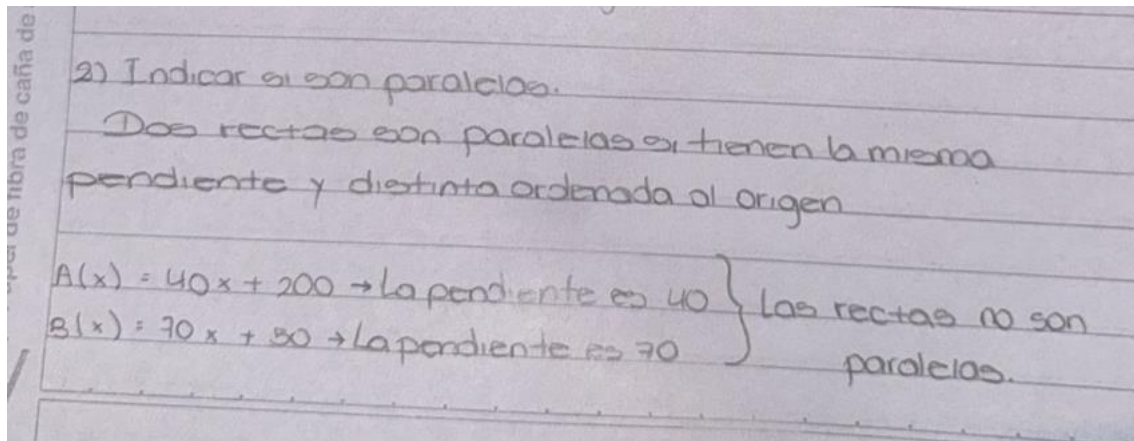
Se trabajó con tres funciones que representan distintos planes de contratación de horas ofrecidos por una empresa de desarrollo de software.

Pendiente y ordenada al origen



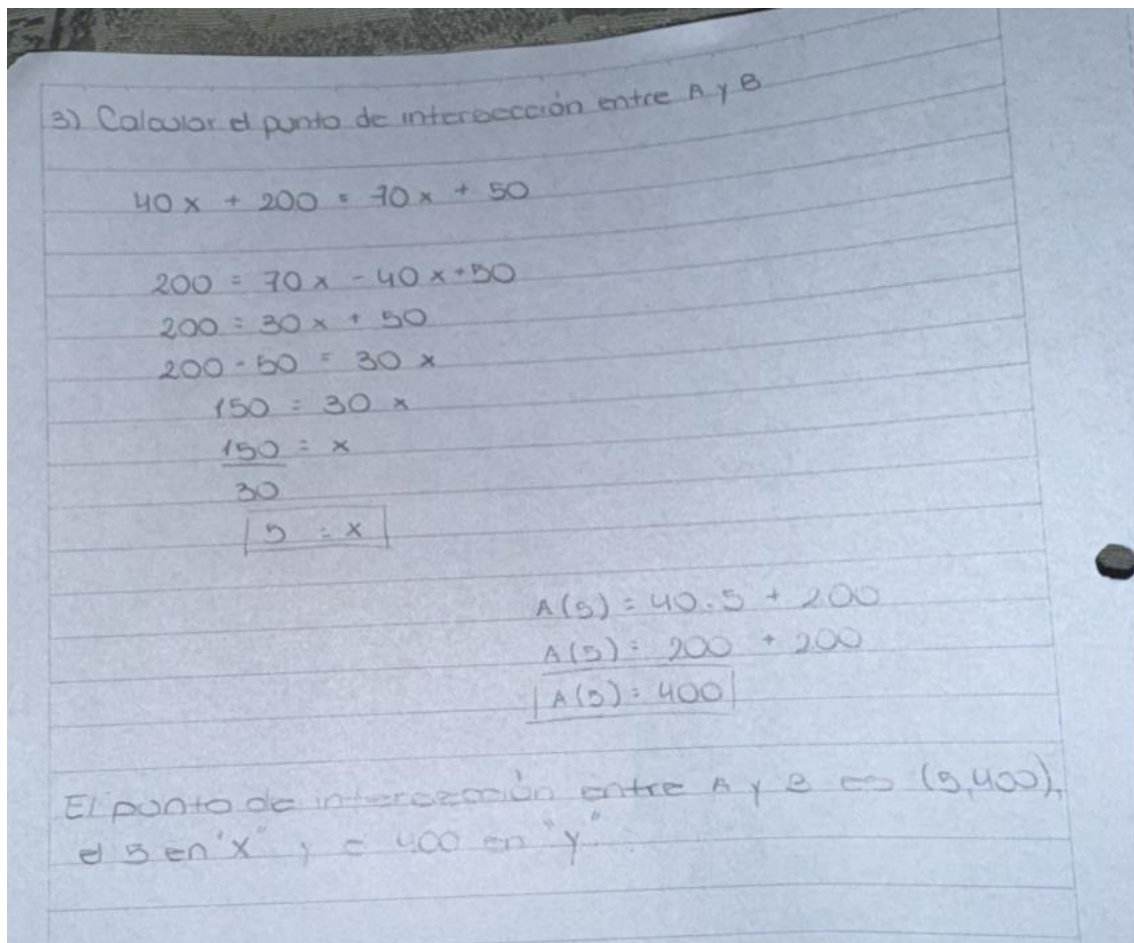
En la función B la pendiente es 70 y la ordenada al origen es 50. Esto indica que el costo aumenta más rápidamente que en el plan A, aunque el costo inicial es menor.

Paralelismo entre funciones



Como las pendientes son diferentes, ambas rectas se cruzan en un punto.

Punto de intersección



Para calcular el punto de intersección entre A y B se igualaron ambas funciones, el resultado fue: (5, 400)

Esto significa que para 5 horas ambos planes tienen el mismo costo.

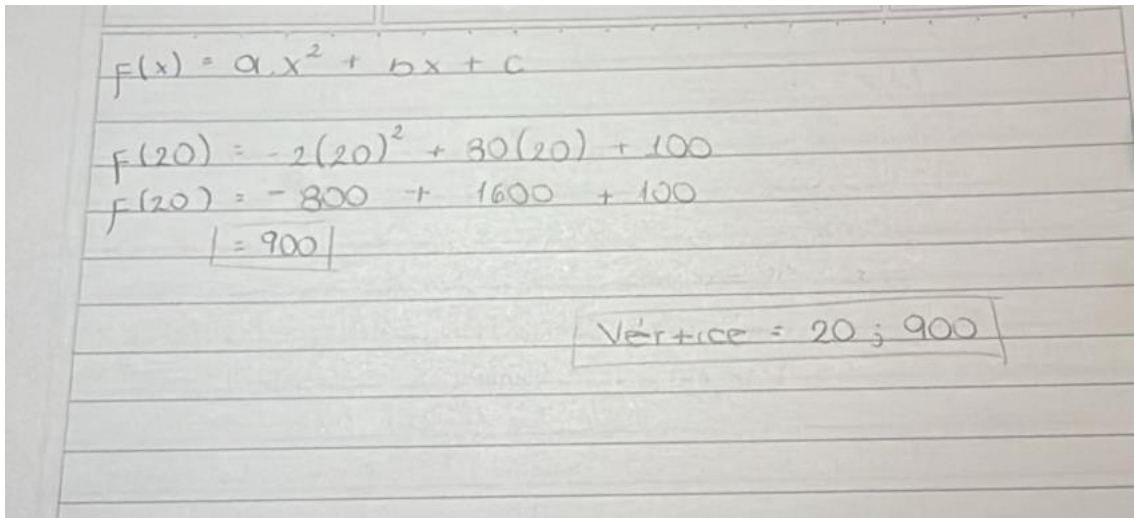
Análisis de la función cuadrática

La función C es una función cuadrática con coeficiente principal negativo.

$$C(x) = -2x^2 + 80x + 100$$

Debido a que el coeficiente de x^2 es negativo, la parábola se abre hacia abajo.

Vértice



Handwritten calculation on lined paper:

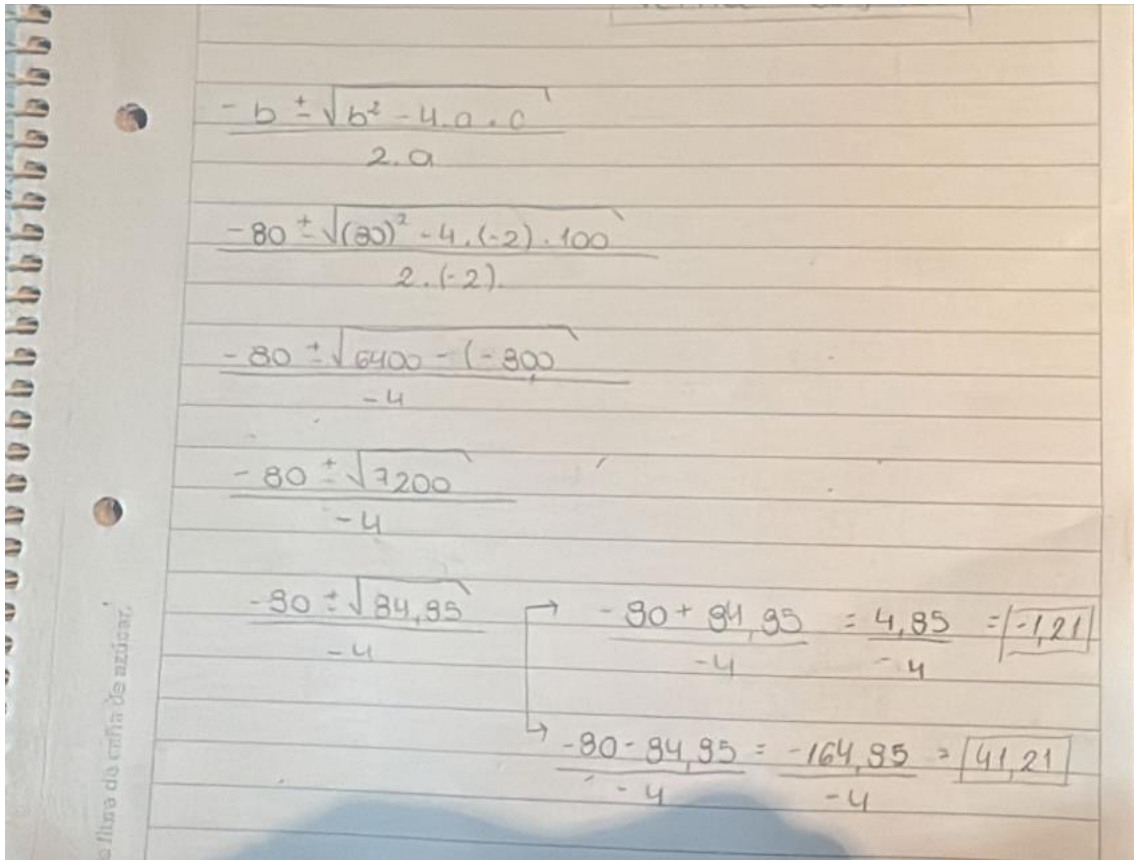
$$F(x) = a \cdot x^2 + b \cdot x + c$$
$$F(20) = -2(20)^2 + 80(20) + 100$$
$$F(20) = -800 + 1600 + 100$$
$$= 900$$

Vértice = 20 ; 900

El vértice es:

(20, 900)

Este punto representa el valor máximo de la función.



Handwritten calculation of the roots of a quadratic equation using the quadratic formula:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

$$= \frac{-80 \pm \sqrt{(80)^2 - 4 \cdot (-2) \cdot 100}}{2 \cdot (-2)}$$

$$= \frac{-80 \pm \sqrt{6400 - (-800)}}{-4}$$

$$= \frac{-80 \pm \sqrt{7200}}{-4}$$

$$= \frac{-80 \pm \sqrt{84,95}}{-4}$$

Two branches are shown for the square root:

$$\rightarrow \frac{-80 + 91,95}{-4} = \frac{11,95}{-4} = -2,9875 \approx -1,21$$

$$\rightarrow \frac{-80 - 91,95}{-4} = \frac{-171,95}{-4} = 42,9875 \approx 41,21$$

Raíces de la función

Los resultados aproximados fueron:

$X_1 \approx 41.21$

$X_2 \approx -1.21$

Dentro del dominio planteado solo resulta útil la raíz positiva.

Parte B

El código del programa:

```

1  try:
2      valores=[0]*9
3      print("COMPARACION DE PLANES")
4      print()
5      print("Ingrese los valores en X: ")
6      for i in range(9):
7          valores[i]=int(input())
8
9      for i in range(9):
10         x = valores[i]
11         A = 40 * x + 200
12         B = 70 * x + 50
13         C = -2 * (x * x) + 80 * x + 100
14         print("x =", x)
15         print("Plan A =", A)
16         print("Plan B =", B)
17         print("Plan C =", C)
18
19         if A == B and A < C:
20             print("Planes mas economicos: A y B")
21         elif A < B and A < C:
22             print("Plan mas economico: A")
23         elif B < A and B < C:
24             print("Plan mas economico: B")
25         elif C < 0:
26             print("El Plan C tiene costo negativo")
27             print("No tiene sentido en un caso real")
28             if A < B:
29                 print("Plan mas economico: A")
30             else:
31                 print("Plan mas economico: B")
32         else:
33             print("Plan mas economico: C")
34         print()
35
36     matriz = [[" "]* 60 for i in range(20)]
37
38     for x in range(20):
39         y = 19 - x
40         if y >= 0:
41             matriz[y][x * 2] = "A"
42
43     for x in range(20):
44         y = 19 - x
45         if y >= 0 and x + 15 < 60:
46             matriz[y][x + 15] = "B"
47
48     for x in range(20):
49         y = ((x - 10) * (x - 10)) // 10
50         y = y + 11
51         if y < 20 and x + 35 < 60:
52             matriz[y][x + 35] = "C"
53
54     print()
55     print("GRAFICO EN CONSOLA")
56     print()
57     for fila in range(20):
58         for columna in range(60):
59             print(matriz[fila][columna], end="")
60         print()
61     print()
62     print("A = Plan A")
63     print("B = Plan B")
64     print("C = Plan C")
65 except ValueError:
66     print("Error")

```

El programa permite:

Evaluar las funciones para distintos valores de x.

Calcular el costo de cada plan.

Determinar el plan más económico.

Detectar valores negativos en la función C.

Representar gráficamente las funciones mediante una matriz en consola.

Evaluación de funciones

Se trabajó con el siguiente conjunto de valores:

0, 5, 10, 15, 20, 25, 30, 40 y 50

Para cada valor el programa calcula automáticamente los costos de los tres planes.

Posteriormente compara los resultados para determinar cuál posee el menor costo.

Representación gráfica

Debido a que no se utilizaron librerías gráficas externas, las funciones fueron representadas mediante una matriz en consola utilizando caracteres.

La matriz fue creada con 20 filas y 60 columnas.

Dentro de la matriz:

La letra "A" representa el gráfico del Plan A.

La letra "B" representa el gráfico del Plan B.

La letra "C" representa el gráfico del Plan C.

Cada función fue dibujada utilizando coordenadas calculadas manualmente.

Las funciones lineales fueron representadas mediante diagonales ascendentes y la función cuadrática mediante una parábola.

De esta forma se logró una representación visual aproximada del comportamiento de las funciones dentro del intervalo analizado.

Parte C

A partir de los resultados obtenidos se pudo analizar qué plan resulta más conveniente según la cantidad de horas contratadas.

El Plan B resulta más económico para cantidades pequeñas de horas debido a su bajo costo inicial.

Sin embargo, el Plan A posee una pendiente menor, por lo que aumenta más lentamente y termina siendo más conveniente para valores mayores de horas.

Ambos planes poseen el mismo costo cuando:

$$X = 5$$

La función C presenta un comportamiento distinto debido a que es cuadrática.

Inicialmente el costo aumenta, alcanza un valor máximo y posteriormente comienza a disminuir.

A partir de cierto valor de x , la función comienza a producir costos negativos.

Esto representa un problema real, ya que un costo negativo no tiene sentido en el contexto de contratación de servicios, al cliente se le dejaría de cobrar y la empresa dejaría de obtener ganancias.

Por este motivo se concluye que, aunque la función matemáticamente es correcta, deja de representar adecuadamente una situación real para valores elevados de horas.

CONSIGNA 3

PARTE A

1- *Cada fila representa una función del sistema

-Fila 1= representa la autenticación

-Fila 2= representa el procesamiento de datos

-Fila 3= Representa la generación de reportes

*Cada columna representa un servidor distinto

-Columna 1= primer servidor

-Columna 2= segundo servidor

-Columna 3= cuarto servidor

*Cada valor indica el tiempo promedio de ejecución de una función en un servidor determinado -Por ej, en la fila 3 columna 3, está el numero 95, que determina que la función 3 tarda 95 ms en promedio para ejecutarse

2- La matriz M y la matriz C tienen ambas una dimensión de 3x3, es posible realizar el producto de ambos debido a que la cantidad de columnas de la matriz M es igual a la cantidad de filas de la matriz C

PARTE B

3-

1= Sumar el tiempo promedio de respuesta en ms de la misma función que se representa por filas, dentro de todos los servidores.

Tiempo promedio de ejecución por función= $(120+150+100)/3=123.33$ (función 1)

$(200+180+220)/3=200$ (función 2) $(90+110+95)/3=98.33$ (función 3)

2=Sumar el tiempo promedio de respuesta en ms de todas las funciones que hay en un solo servidor, el cuál se representa en columnas

Tiempo promedio de ejecución por servidor= $(120+200+90)/3=136.67$ (servidor 1)

$(150+180+110)/3=146.67$ (servidor 2) $(100+220+95)/3=138.33$ (servidor 3)

CÓDIGO EN PYTHON PARA HACER ESTO=

```

1 try:
2     M=[[0]*3 for i in range(3)]
3     print("Ingrese los números de la matriz (ingresando los elementos por fila de izquierda a derecha): ")
4     for i in range(3):
5         for j in range(3):
6             M[i][j]=int(input())
7     for i in range(3):
8         suma=0
9         for j in range(3):
10            suma+=M[i][j]
11        prom=suma / 3
12        print("Promedio función", i+1, "=", prom)
13    for j in range(3):
14        suma=0
15        for i in range(3):
16            suma+=M[i][j]
17        prom=suma / 3
18        print("Promedio servidor", j+1, "=", prom)
19
20    print()
21    MT = [[0]*3 for i in range(3)]
22    for i in range(3):
23        for j in range(3):
24            MT[i][j] = M[j][i]
25    print("MATRIZ TRANSPUESTA: ")
26    for i in range(3):
27        for j in range(3):
28            print(MT[i][j], end=" ")
29        print()
30 except ValueError:
31     print("Error")

```

4- Cuando haces la transpuesta las filas pasan a representar los servidores y las columnas representan las funciones.

$$\begin{pmatrix} 120 & 200 & 90 \\ 150 & 180 & 110 \\ 100 & 220 & 95 \end{pmatrix}$$

PARTE C

5- Calculamos el producto $T = M * C$

$M * C =$

$$\begin{pmatrix} 120 & 150 & 100 \\ 200 & 180 & 220 \\ 90 & 110 & 95 \end{pmatrix} * \begin{pmatrix} 30 & 20 & 10 \\ 15 & 25 & 20 \\ 40 & 10 & 30 \end{pmatrix} = T = \begin{pmatrix} 9850 & 7150 & 7200 \\ 17500 & 10700 & 12200 \\ 8150 & 5500 & 5940 \end{pmatrix}$$

PARTE D

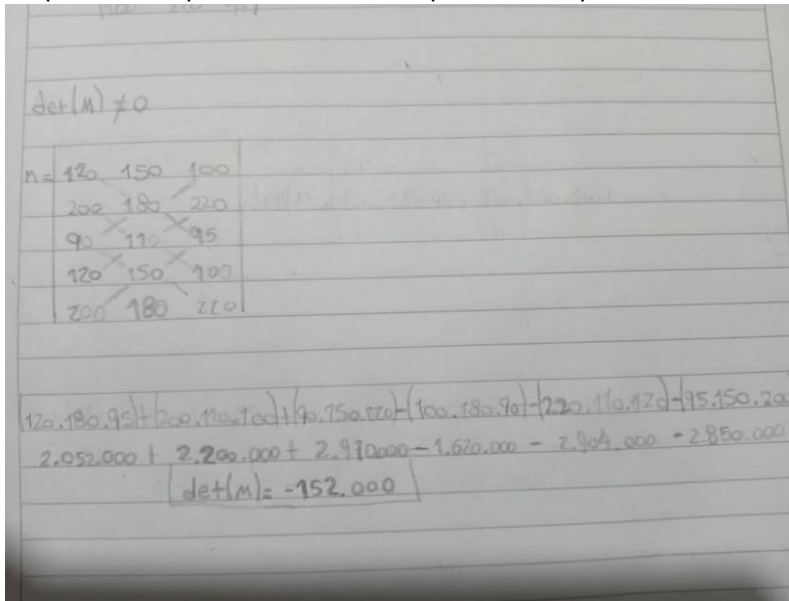
6- Cada valor de T combina datos de funciones y servidores completamente distintos, por lo que no tiene sentido el realizar esta operación

7- Una alternativa más útil sería realizar el producto de Hadamard (multiplicar elementos correspondientes entre si, en este caso, multiplicaríamos el tiempo promedio por la cantidad de ejecuciones, que nos daría como resultado el tiempo total que estuvo cada servidor ejecutando cada función)

PARTE E

8-En este caso la matriz M con su transpuesta M^T no son simétricas. Una matriz es simétrica cuando es igual a su transpuesta, tiene el mismo numero de filas y de columnas.

$$M = \begin{pmatrix} 120 & 150 & 100 \\ 200 & 180 & 220 \\ 90 & 110 & 95 \end{pmatrix} \neq M^T = \begin{pmatrix} 120 & 200 & 90 \\ 150 & 180 & 110 \\ 100 & 220 & 95 \end{pmatrix}$$



Handwritten calculation of the determinant of matrix M:

$$\det(M) = 120 \cdot 180 \cdot 95 + 200 \cdot 110 \cdot 100 + 90 \cdot 220 \cdot 150 - (100 \cdot 180 \cdot 90 + 220 \cdot 150 \cdot 120 + 95 \cdot 200 \cdot 110)$$

$$\det(M) = 2.052.000 + 2.200.000 + 2.970.000 - 1.620.000 - 2.880.000 - 2.090.000$$

$$\det(M) = -152.000$$

Es invertible porque $-152.000 \neq 0$

10 - Si una matriz no es invertible, significa que la información que representa no es completamente independiente, ya que existe dependencia entre sus datos.

PARTE F

11 - La función que más costo computacional promedio representa es la función 2 = (200) Y el servidor que resulta más eficiente es el servidor 1 = (136.67)

12 - La función 2 mostró el mayor costo computacional promedio (200 ms), por lo que habría que optimizar su implementación Al ser el servidor 2 el que mejor realiza la función de mayor costo computacional, se deberían redistribuir las cargas y enviar esta solicitud a este servidor con más frecuencia y repartir más instrucciones 1 y 3 a los demás servidores (por ej: la función 1 al servidor 3 y la función 3 al servidor 1, que es donde tuvieron mejor desempeño)